Insecure by Design: A Hands-On Learning Platform for Web Security Training

Progress Report 3

CSIS 4495-003 | Applied Research Project

Winter 2026

Instructor: Arasanipalai Kandhadai, Padmapriya

Allwyn Mascarenhas | 300381351 | Team Leader

Thilan Udahage Don | 300398798

Table of Contents

Introduction 3
Summary of Initially Proposed Project 4
Changes to the Proposal 4
Adjustment to Vulnerability Scope 4
Project Planning and Timeline 5
Project Timeline 5
Team Responsibilities 5
Project Management 6
Updated Features – Progress Report 2 7
1. Student Performance Tracking 7
AI Use Section 11
Table of AI Tools and Specific Use 13
Appendix 13
Work Log 13
References 17

Introduction

Web application security remains a critical area of study as modern systems increasingly rely on complex, interconnected web technologies. While theoretical resources exist, many learners struggle to understand how vulnerabilities emerge in real workflows without hands-on experimentation. Practical exposure is essential for developing the ability to recognize, exploit, and mitigate security flaws; skills foundational to cybersecurity, software development, and digital forensics.

Existing vulnerable training platforms such as DVWA, WebGoat, and OWASP Juice Shop provide valuable learning opportunities, but they often rely on outdated architectures or lack structured guidance. As noted in the proposal, "many rely on outdated architectures, lack structured guidance, or do not reflect the design patterns and frameworks commonly used in contemporary enterprise applications."

Industry research reinforces this need. According to Fortinet's 2024 Cybersecurity Skills Gap Report, "58% of IT decision-makers identify a lack of hands-on cybersecurity skills and training among staff as the leading cause of security breaches."

This project addresses that gap by designing an intentionally vulnerable Spring Boot MVC application that embeds realistic vulnerabilities into functional workflows. The research explores:

- How intentional vulnerabilities can be embedded safely and educationally
- Whether guided exploration improves understanding of secure coding
- Which vulnerabilities are most effective for foundational learning

The initial hypothesis is that interacting with vulnerabilities in realistic workflows improves comprehension, engagement, and retention.

Summary of Initially Proposed Project

The original project proposed the creation of Insecure by Design, a modern, intentionally vulnerable web application built using Spring Boot MVC. The goal was to provide a hands-on learning platform that embeds common vulnerabilities into realistic workflows, enabling learners to explore, exploit, and understand security flaws in a safe environment.

The proposal outlined:

- A design-and-implementation research methodology
- A modern technology stack (Spring Boot, H2/MySQL, Thymeleaf, HTML/CSS/JS)
- A set of planned vulnerabilities including SQL Injection, XSS, Broken Authentication, CSRF, IDOR, and File Upload flaws
- A structured timeline from Week 1 to Week 13
- Clear division of responsibilities between backend and frontend roles

Changes to the Proposal

At this stage of the project, the core direction remains aligned with the original proposal. However, the following adjustments were made:

Adjustment to Vulnerability Scope

During development, we realized that the XSS and SQL Injection modules required more time than originally estimated, particularly in designing realistic workflows, implementing safe and vulnerable modes, and ensuring the educational clarity of each demonstration. To maintain quality and avoid delivering a partially implemented feature, we decided to defer the Broken Authentication module to the post-midterm phase. This adjustment allows us to deliver two fully functional, polished, and academically effective modules for the midterm, while still keeping the overall project aligned with the original proposal.

Project Planning and Timeline

Project Timeline

Week	Milestones	Deliverables
Week 7 — Feb 24	Implement additional vulnerabilities (Reflected XSS); Conduct internal testing; Record midterm demo video	Midterm Report + Video Archive + Working Implementation (Due Feb 23, 11:59 PM)
Week 8 — Mar 3	Implement Stored XSS; Broken Auth	Progress Report + Research/Implementation 2 (Due Mar 2, 11:59 PM)
Week 9 — Mar 10	Implement CSRF vulnerability; Improve UI/UX	Progress Report + Research/Implementation 3 (Due Mar 9, 11:59 PM)
Week 10 — Mar 17	Implement IDOR and File Upload vulnerability; Begin writing vulnerability walkthroughs	Progress Report + Research/Implementation 4 (Due Mar 16, 11:59 PM)
Week 11 — Mar 24	Full system testing; Finalize all vulnerabilities; Begin drafting final report	Progress Report + Research/Implementation 5 (Due Mar 23, 11:59 PM)
Week 12 — Mar 31	Finalize documentation; Prepare final presentation slides; Clean and organize GitHub repo	
Week 13 — Apr 7	Deliver final defense presentation; Submit final report and implementation	Final Report + Implementation (Due Apr 8, 11:59 PM)

Team Responsibilities

<u>Allwyn Mascarenhas (Team Lead)</u>

- Backend architecture and Spring Boot implementation
- Database design (H2/MySQL)
- Backend-driven vulnerabilities (SQL Injection, Broken Authentication, IDOR)
- Midterm and final demo coordination

Thilan Udahage Don

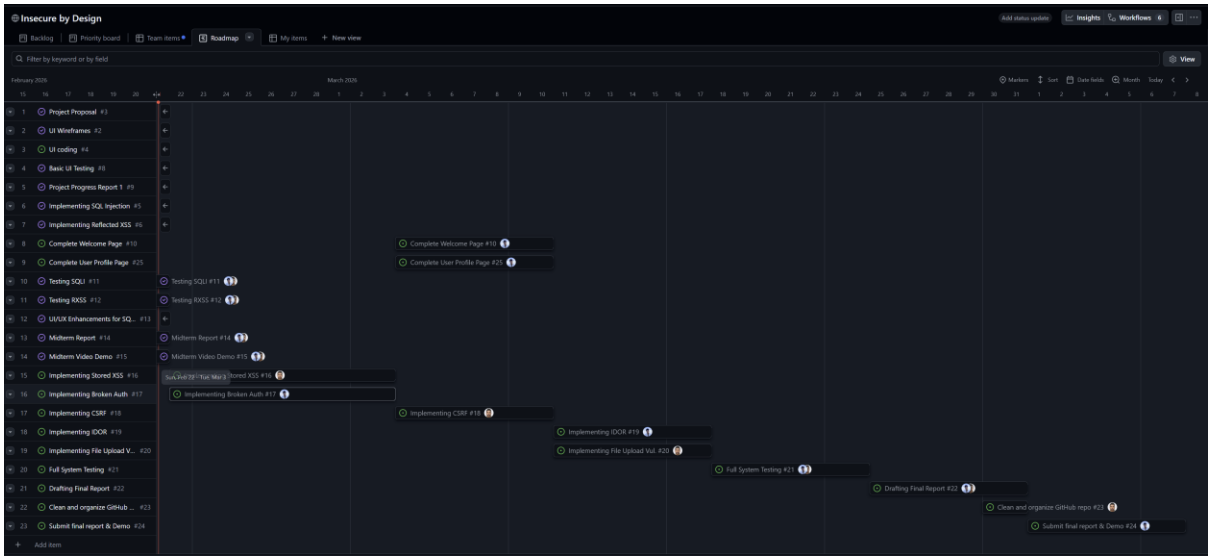
- Frontend development (Thymeleaf, HTML, CSS, JS)
- Guided learning flow and UI/UX
- Frontend-related vulnerabilities (XSS, CSRF, File Upload flaws)
- System testing and vulnerability walkthroughs

Both members collaborate on:

Vulnerability research, GitHub repo management and project board, testing, documentation (proposal, midterm report, final report) video recording, and final presentation.

Project Management

The project tasks will be tracked and assigned using GitHub projects. Tasks that need to be completed are as follows,



Updated Features – Progress Report 3

1. **Student Performance Tracking**

To support learning outcomes and grading, the application was extended with a progress-tracking design that records each student's interaction with vulnerability modules. The system associates progress with the authenticated username from Spring Security and stores progress data in an H2 database. Each module view and submission produces a progress event (view/attempt/completion) and updates a per-module progress record containing status, attempt count, timestamps, and optional scoring. This design enables a "Progress" dashboard showing which modules have been attempted or completed, when completion occurred, and how many attempts were required. The tracking logic is implemented as a centralized ProgressService and instrumented in each module controller (e.g., XSS and SQL Injection) to record module views and evaluate completion conditions based on the student's submitted payload and the module's response behavior. While the app uses in-memory authentication for simplicity, the progress system is database-backed to persist outcomes across sessions and provide an auditable learning history.

- **Module viewed**: student opened the module page (e.g., /sql-injection, /xss).
- **Module completed**: student performed the "success action" for that module.
    - SQL Injection: successfully triggered the vulnerable behavior (e.g., returned >1 row via injection) *and/or* successfully used safe mode to block the injection.
    - XSS: successfully executed a payload in the unsafe section *and/or* confirmed safe rendering and sanitization output.
- **Evidence**: store the input/payload used, whether it worked, and the mode (vuln/safe).
- **Metrics**: first attempt time, last attempt time, number of attempts, completion time, and optional score.

This allows you to show a clear "student journey" in a report and demonstrate you are tracking outcomes, not just page visits.

How progress is captured in the app

1) When a module page loads (GET)

On every module GET endpoint (like /xss, /sql-injection, etc.), log a VIEW event and update student_progress:

- If no row exists → create it with status = IN_PROGRESS

- Update last_seen_at
- Do not mark as completed just for viewing

2) When the student submits a payload (POST/GET with query params)

When the user submits the input for the module (XSS POST, SQLi GET with q + mode), log an ATTEMPT event:

- Increment attempt_count.
- Record payload/parameters
- Determine outcome:
    - For SQLi:
        - If "vuln mode" and payload causes abnormal result set (e.g., returns all users) → mark attempt as SUCCESS
        - If "safe mode" and injection fails (returns no broad results) → mark attempt as BLOCKED (still valuable learning)
    - For XSS:
        - If unsafe rendering shows HTML tags rendered as HTML → treat as "unsafe success"
        - For script execution specifically, you can mark "success" based on known payload patterns (more on this below)
- If success conditions met → set status = COMPLETED and completed_at

3) Provide a progress page

Add a new route:

- /progress (student view): shows modules + completion statuses.
- Optional /admin/progress (instructor view): show all students, completion, and timestamps.

Code changes that would be needed (high-level, no code)

New files/classes

1. **Progress entities (JPA) or JDBC models**

- StudentProgress entity
- Optional ProgressEvent entity

2. **Repositories**

- StudentProgressRepository
- Optional ProgressEventRepository

3. **ProgressService**

   Central service with methods like:

- recordView(username, moduleKey)
- recordAttempt(username, moduleKey, input, result)
- markCompleted(username, moduleKey, score)

4. **ProgressController**

- GET /progress → show current user progress table
- Optional GET /admin/progress → instructor overview

Modified existing controllers (instrumentation)

- XssController:
  - On GET: record view
  - On POST: record attempt (payload), compute result, mark completed if rule matches
- SqlInjectionController:
  - On GET: record view + record attempt if query param present

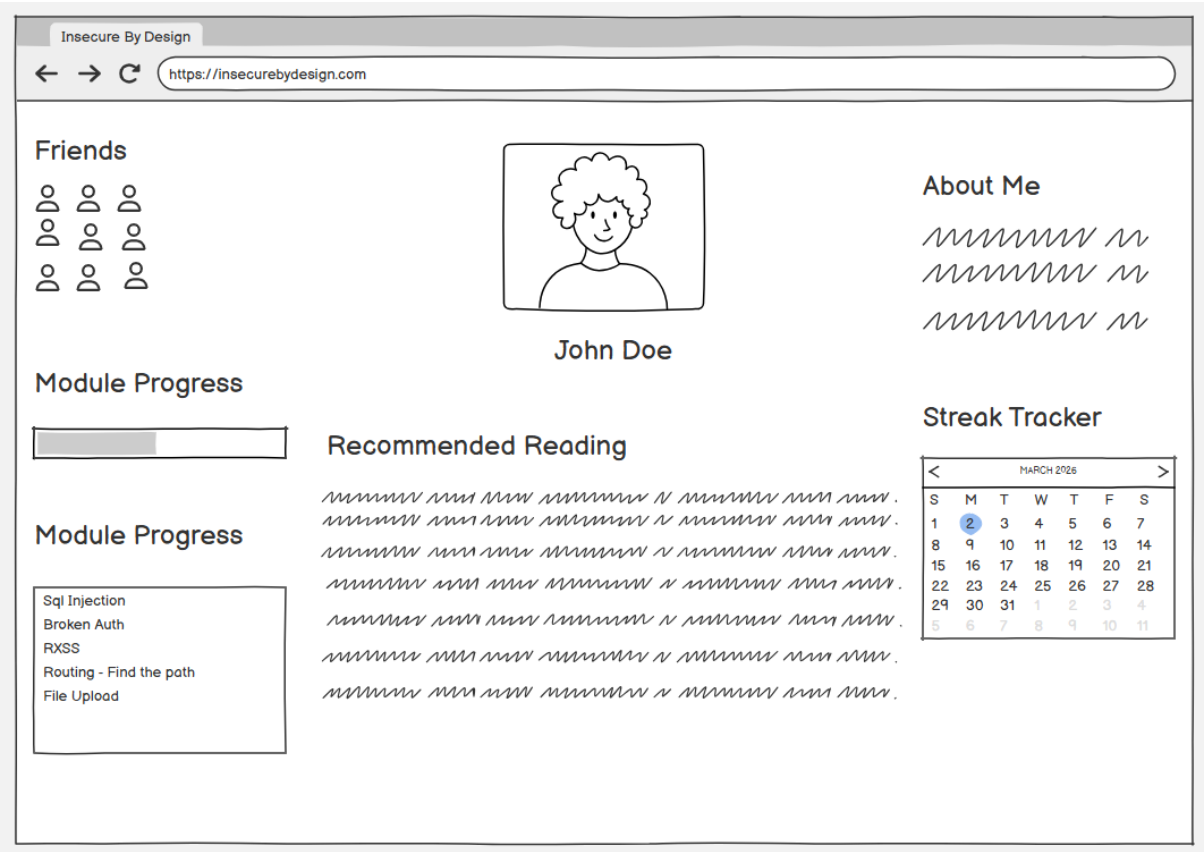
- Other module endpoints later:
    - BrokenAuthController, SecurityMisconfigController, etc.

Database init

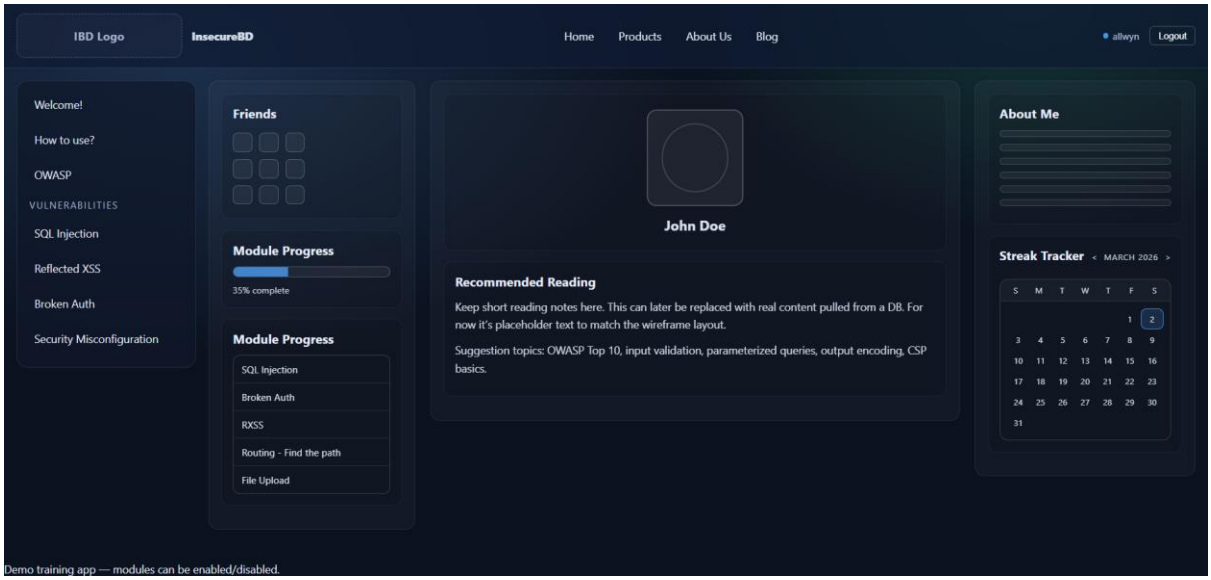
- Add schema.sql or JPA auto-DDL (you already used SQL init for H2 earlier)
- Seed module definitions (optional) if you want a consistent ordering

UI changes

- Add a “Progress” link in the top navbar/sidebar
- Create templates/pages/progress.html with:
    - list of modules
    - status badge (Not started / In progress / Completed)
    - completed timestamps
    - attempts count



User Profile Page



2. **Performance Tracking Backend**

XP & Profile System — Technical Implementation

XP Calculation

- Each challenge level awards a fixed amount of XP upon successful completion (e.g. SQLi has 3 levels worth 200 XP each, totalling 600 XP max)
- The backend tracks which levels a user has completed and sums the corresponding XP values to calculate a total XP score
- XP is stored persistently in a database linked to the user's account, so progress is saved between sessions

Rank System

- Ranks are assigned based on XP thresholds (e.g. 0–299 = Beginner, 300–599 = Apprentice, etc.)
- A server-side method compares the user's total XP against these thresholds to determine their current rank and what the next rank is
- The percentage progress toward the next rank is calculated using: (currentXP - currentRankMin) / (nextRankMin - currentRankMin) * 100 and passed to the frontend to render the progress bar width dynamically via Thymeleaf (th:style)

Badge Logic

- Badge eligibility is determined by checking boolean flags on the user's progress object (e.g. sqliLevel1Completed, sqliLevel2Completed, sqliLevel3Completed)

- If all levels in a category are marked true, the corresponding badge flag (e.g. sqlieBadgeEarned) is set to true
- Thymeleaf conditionally applies CSS classes (earned or locked) to each badge card based on these flags, changing the visual styling accordingly

Progress Tracking

- A Progress object (or database entity) is associated with each user and contains boolean fields for every individual challenge level
- When a user successfully completes a level, the backend updates the relevant boolean field to true and recalculates XP and badge status
- The controller retrieves this progress object and passes it to the Thymeleaf template as a model attribute, where it is used to render the dots as green (done) or grey

Frontend Rendering (Thymeleaf)

- th:text is used to dynamically display values like username, XP total, rank name, and XP remaining to next rank
- th:classappend conditionally adds CSS classes to elements (e.g. adding done to a level dot, or earned to a badge card) without overwriting existing classes
- th:style injects the calculated rank progress percentage directly into the progress bar's inline CSS width property

AI Use Section

Table of AI Tools and Specific Use

AI Tool	Version	Feature Used	Value Add
Chatgpt	5.1 free	Grammar and writing help	Used only as a writing aid to flesh out original ideas
Claude	4.5 free	Prototyping and coding	Used to generate quick mocks before making final commitment to the design.

Appendix

Work Log

Allwyn Mascarenhas

Date	Number of Hours	Description of Work
15th Jan	2	Conducted initial research on OWASP Top 10 vulnerabilities and reviewed documentation for DVWA, WebGoat, and Juice Shop. Compared their learning approaches and identified opportunities for a simpler, framework-based platform.
20th Jan	1	Participated in instructor-led project consultation. Discussed project direction, technology stack, and how to embed vulnerabilities safely within a Spring Boot MVC application.
21st Jan	1.5	Co-drafted the pre-proposal for instructor approval. Summarized research findings and outlined the initial problem definition, research questions, and project scope.
22nd Jan	1	Researched Spring Boot security misconfigurations and common vulnerability patterns (e.g., insecure authentication flows, improper input validation). Documented potential implementation strategies.
23rd Jan	1	Attended scheduled Friday team meeting. Finalized task distribution and reviewed proposal template.

Date	Number of Hours	Description of Work
23rd Jan	1.5	Created and organized the GitHub Project Kanban board. Added tasks for proposal writing, research, vulnerability implementation, and documentation. Assigned responsibilities and set up workflow columns.
24th Jan	2	Collaborated with Thilan on drafting the full project proposal. Wrote sections on scope, methodology, and technology stack. Ensured the proposal met the rubric requirements for introduction and research design.
25th Jan	2	Continued refining the proposal independently. Reviewed the draft for clarity, strengthened the methodology section, and added additional justification for the selected technology stack.
26th Jan	1	Reviewed and refined the proposal draft. Added details to the methodology and expected results sections. Submitted the document to Blackboard and GitHub.
27th Jan	1	Participated in in-class proposal check-in. Instructor requested adding industry data to justify the project. Reviewed Fortinet's 2024 Skills Gap Report and incorporated the statistic that 58% of IT decision-makers cite lack of hands-on skills as a top cause of breaches.
31st Jan	1	Attended scheduled Friday team meeting. Discussed Week 5 implementation tasks, including setting up the Spring Boot skeleton and configuring the H2 database.
1st Feb	1.5	Attended scheduled Saturday team meeting. Reviewed architecture outline and refined backend workflow diagrams. Finalized vulnerability placement strategy for SQL Injection, Broken Authentication, and IDOR.
3rd Feb	1	Began setting up backend project structure. Created initial controller and service templates to prepare for SQL Injection implementation.
4th Feb	1.5	Completed Spring Boot skeleton setup. Verified project builds successfully, configured H2 database, and tested basic backend routing.

Date	Number of Hours	Description of Work
5th Feb	1	Conducted deeper research on SQL Injection exploitation in Java applications. Reviewed insecure query patterns and prepared notes for implementing the first vulnerability.
6th Feb	1	Drafted backend-focused content for Progress Report 1, summarizing architecture planning, backend setup, and SQL Injection research.
7th Feb	1	Updated backend folder structure and added initial placeholder files for SQL Injection implementation. Reviewed vulnerability placement documentation added by Thilan.
9th Feb	2	Implemented the vulnerable SQL query using string concatenation. Verified that SQLi payloads produced unsafe behavior and returned unintended results.
10th Feb	1.5	Implemented the safe SQL query using prepared statements. Ensured parameterized queries blocked injection attempts and added backend logic to toggle between safe and vulnerable modes.
13th Feb	1	Weekly team meeting. Reviewed SQLi progress, discussed RXSS routing improvements, and planned controller refactoring for better modularity.
14th Feb	1	Continued team meeting. Finalized SQLi backend logic and confirmed the structure for GET-based reflection in the upcoming RXSS controller.
15th Feb	1	Added backend support for executed SQL query display and improved error handling for malformed SQL payloads. Ensured the application returned meaningful error messages without breaking.
17th Feb	1	Created a dedicated RxssController to separate XSS logic from other modules. Cleaned up routing and ensured proper GET-based reflection behavior.
18th Feb	1	Adjusted the CSP header from Content-Security-Policy to Content-Security-Policy-Report-Only to allow XSS payload execution for demonstration while still logging violations.
20th Feb	1	Performed integration testing for SQLi and RXSS modules. Verified backend compatibility with UI

Date	Number of Hours	Description of Work
		components, dropdown behavior, and scroll helper functionality.
21st Feb	1.5	Wrote backend sections of SQLi implementation details and RXSS controller explanation. Participated in recording the midterm demonstration video. Uploaded updated code, the completed midterm report, and the demonstration video to the GitHub repository.
22nd Feb	2	Designing and coding the student tracking page.
24th Feb	2	Coding layout for user profile page
26th Feb	2	Creating and testing user profile controller, css, html.
1st March	2	Reading and research for broken auth vuln deployment.
3rd March	2	Student tracking background coding
5th March	1	UI redesign call with team member to create wire frames

References

Fortinet. (2024). 2024 Cybersecurity Skills Gap Report. Retrieved January 2026, from https://www.fortinet.com/content/dam/fortinet/assets/reports/2024-cybersecurity-skills-gap-report.pdf

Mudge, R. (digininja). (n.d.). Damn Vulnerable Web Application (DVWA) [GitHub repository]. Retrieved January 2026, from https://github.com/digininja/DVWA

OWASP Foundation. (n.d.). OWASP Juice Shop. Retrieved January 2026, from https://owasp.org/www-project-juice-shop/

OWASP Foundation. (n.d.). OWASP WebGoat. Retrieved January 2026, from https://owasp.org/www-project-webgoat/